

FlexiStaff

UN PROFIL, TOUS LES MÉTIERS

# De la photo au modèle en production

## Guide de l’étudiant

Fabio Cassiano

Version 1.1, 2026-11-13

# Formation Développeur IA · École IA Brest

## **Cas pratique FlexiStaff** : Vision, OCR & intégration CalDAV

**Intervenant**

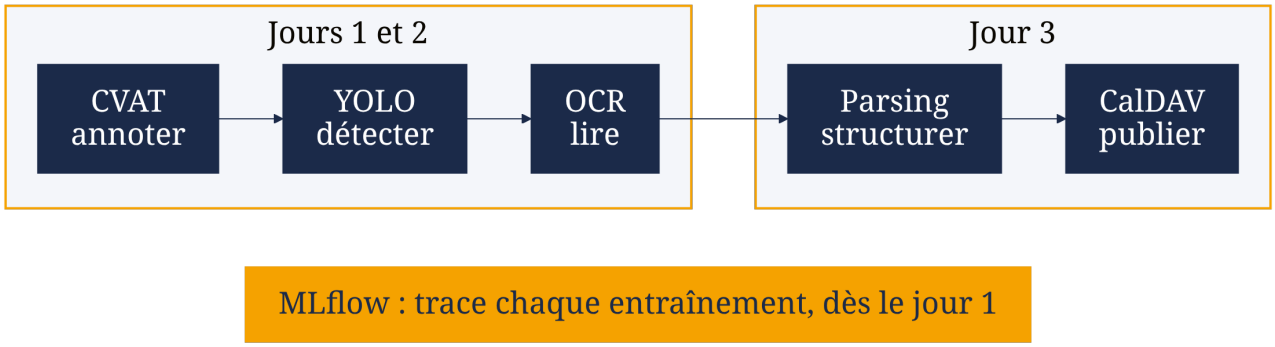
**Fabio Cassiano**
Ingénieur de recherche en IA · Exail (Brest)
fabio.cassiano@hotmail.fr

| | |
|---|---|
| **Durée** | 3 jours (9h00 - 17h00) |
| **Format** | Squads de 3 ou 4 étudiants |
| **Version** | 1.1 - 2026-11-13 |

### Les trois jours

| Jour | Ce que vous faites | Ce que vous livrez |
|---|---|---|
| J1 | Labelliser un jeu de données, le préparer, lancer un premier entraînement YOLO **tracé dans MLflow** | Guide de labellisation, jeu de données labellisé, premier run MLflow, premier modèle enregistré (« champion ») |
| J2 | Tester la lecture OCR sur le champion, évaluer, améliorer, comparer des challengers au champion et promouvoir le meilleur | Challenger(s) comparé(s) au champion, test OCR, promotion justifiée |
| J3 | Écrire le parsing de la sortie OCR, publier les événements dans l'agenda (CalDAV), voir votre champion dans l'application CalEvent | Parser testé, événements publiés sans doublon, démonstration |

### La chaîne complète



# Sommaire

1. Comment utiliser ce guide . . . 3
2. Le cas pratique . . . 3
3. Infrastructure . . . 4
   3.1. Architecture déployée . . . 4
   3.2. Tester sa connexion . . . 4
   3.3. Le portail . . . 5
4. Organisation du travail . . . 5
5. Jour 1 : labelliser, entraîner et tracer un premier modèle . . . 6
   5.1. Objectifs . . . 6
   5.2. Labellisation avec CVAT . . . 6
   5.3. Préparer le jeu de données . . . 8
   5.4. Tracer avec MLflow dès le premier entraînement . . . 8
   5.5. Premier entraînement YOLO . . . 10
   5.6. Enregistrer votre premier champion (Registry) . . . 11
   5.7. Livrables du jour 1 . . . 11
   5.8. Checklist de fin de journée . . . 12
6. Jour 2 : évaluer, améliorer, comparer, promouvoir . . . 13
   6.1. Objectifs . . . 13
   6.2. Partir du run de la veille . . . 13
   6.3. Tester l'OCR sur le champion . . . 13
   6.4. Évaluer son modèle . . . 13
   6.5. Améliorer . . . 14
   6.6. Ajouter des données simulées . . . 14
   6.7. Comparer les runs . . . 15
   6.8. Challenger et champion . . . 15
   6.9. Bonus : et sans IA ? . . . 15
   6.10. Livrables du jour 2 . . . 17
   6.11. Checklist de fin de journée . . . 17
7. Jour 3 : du texte lu à l'agenda . . . 17
   7.1. Objectifs . . . 17
   7.2. Point de départ . . . 18
   7.3. Parsing défensif . . . 18
   7.4. Publier dans l'agenda (CalDAV) . . . 19
   7.5. L'application de démonstration : CalEvent . . . 20
   7.6. Bonus recommandé : mettre votre champion en production . . . 20
   7.7. Répartition du travail . . . 21
   7.8. Livrables du jour 3 . . . 21
   7.9. Checklist de fin de journée . . . 21
8. Dépannage (FAQ) . . . 22
9. Glossaire . . . 22
10. Ressources . . . 23

# 1. Comment utiliser ce guide

Ce guide vous accompagne pendant les trois jours. Il ne remplace pas le support de cours : il contient ce dont vous avez besoin **les mains sur le clavier** (adresses, commandes, consignes, livrables, dépannage).

- Chaque jour a ses **objectifs**, ses **étapes**, ses **livrables** et une **checklist de fin de journée**.
- Les adresses des services et les commandes sont à copier-coller telles quelles.
- En cas de blocage, consultez la section Dépannage en fin de document : la solution s'y trouve peut-être. Sinon, n'hésitez pas à demander.



Vos identifiants (CVAT, MLflow, RustFS, et le compte Radicale de votre squad) se trouvent sur votre profil du portail. Ne les écrivez jamais dans un fichier versionné (Git), un notebook partagé ou un canal public.

# 2. Le cas pratique

**Le client** : FlexiStaff, agence d'intérim (logistique, santé, BTP) : 12 agences, 340 intérimaires en mission, 47 entreprises clientes et autant de formats de planning que de clients.

**Le problème** : Sophie, gestionnaire de planning à Brest, reçoit chaque semaine une vingtaine de plannings (photos prises au téléphone, scans de tableaux Excel annotés à la main, PDF) et ressaisit tout manuellement :

- 14 h par semaine et par agence de ressaisie ;
- 1 créneau sur 12 comporte une erreur ;
- 48 h de délai moyen avant qu'un changement soit répercuté.

**La mission** : une application qui **photographie** un planning, en **extrait** les créneaux et les **propose** à l'ajout dans l'agenda partagé de l'agence.

| Dans le périmètre | Hors périmètre |
|---|---|
| ✔ Détecter les zones utiles sur la photo<br>✔ En extraire le texte<br>✔ Reconstituer des créneaux structurés<br>✔ Les publier dans l'agenda (CalDAV) | ✖ Développer l'application mobile<br>✖ Entraîner un moteur OCR |



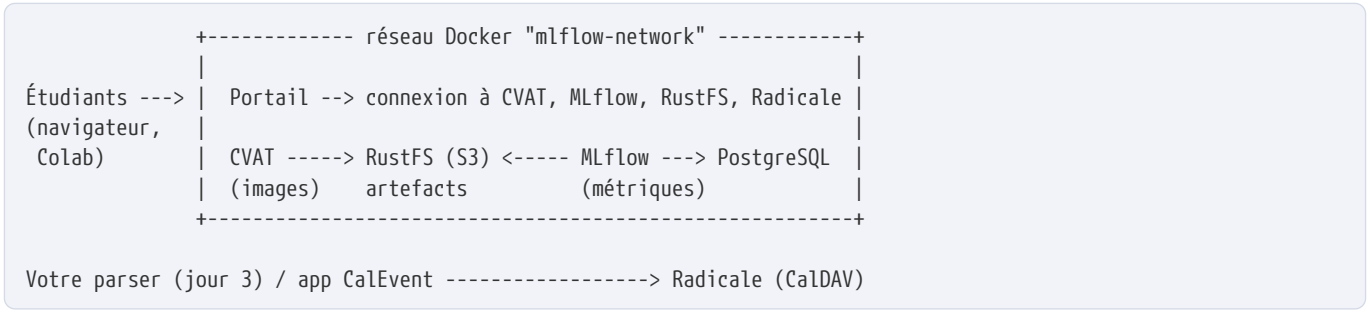
Chaque brique est simple. Le métier, c'est de les faire tenir ensemble. Dans votre squad, vous tenez **tous** les rôles : data engineer, annotateur, ML engineer, MLOps, développeur IA.

# 3. Infrastructure

Tous les services tournent sur **une VM hébergée sur AWS**. Son adresse : `13.37.102.106`

| Service | Rôle | Port | URL |
|---|---|---|---|
| Portail | Connexion unique, identifiants, accès aux outils | 80 | `http://13.37.102.106/` |
| CVAT | Annotation des images | 8080 | `http://13.37.102.106:8080` |
| MLflow | Suivi des entraînements et registre de modèles (protégé par le portail) | 5000 | `http://13.37.102.106:5000` |
| RustFS (console) | Stockage S3, interface web | 9001 | `http://13.37.102.106:9001` |
| RustFS (API) | Stockage S3, API | 9000 | `http://13.37.102.106:9000` |
| Radicale | Serveur CalDAV (agenda) | 5232 | `http://13.37.102.106:5232` interface : `http://13.37.102.106:5232/.web/` |
| PostgreSQL | Base de données de MLflow | - | interne, inaccessible |
| CalEvent | Application de démonstration (jour 3, ouverte depuis le portail) | 8000 | `http://13.37.102.106:8000` version mobile : `/app`, API : `/docs` |

## 3.1. Architecture déployée




| | |
|---|---|
| Portail | Votre point d'entrée : connexion, identifiants, équipe, accès aux outils |
| CVAT | Vous y labellisez ; les images viennent du bucket RustFS |
| MLflow | Chaque entraînement = un « run » (paramètres, métriques, fichiers) |
| PostgreSQL | Stocke les paramètres et métriques des runs |
| RustFS | Stocke les gros fichiers (`best.pt`, graphiques) : les artefacts |
| Radicale | L'agenda où l'application publie les événements |

## 3.2. Tester sa connexion

Depuis votre poste (ou une cellule Colab préfixée par `!`) :

```
curl -I http://13.37.102.106/              # Portail
curl -I http://13.37.102.106:8080          # CVAT
curl    http://13.37.102.106:5000/health   # MLflow -> 401 (identifiants requis)
curl    http://13.37.102.106:9000/health   # RustFS -> {"status":"ok",...}
curl -I http://13.37.102.106:5232/.web/    # Radicale
```

Toute réponse (même 302 ou 401) prouve que le service répond ; sinon, prévenez l'intervenant.

### 3.3. Le portail

Le portail, `http://13.37.102.106/`, est votre point d'entrée : un seul identifiant pour tous les outils.

1. **Première connexion** : cliquez sur « Première connexion ? » et saisissez **seulement votre identifiant**, de la forme `prenom.n` (prénom, point, initiale du nom, en minuscules et sans accent). Cette activation ne fonctionne qu'une fois : ensuite, le mot de passe est obligatoire.
2. **Vos identifiants** : votre profil affiche votre mot de passe et les adresses des outils. Téléchargez-les aussi en JSON (`identifiants-<login>.json`) : ce fichier sert à connecter vos notebooks à MLflow (voir jour 1).
3. **Votre squad** : choisissez l'un des trois Poké Bowls, qui révèle votre équipe. Le choix est **définitif**.
4. **Le chef d'équipe** : votez pour un membre de votre squad (vous compris). Le premier à obtenir la majorité est élu : il définit les labels dans CVAT.

Le tableau de bord donne ensuite accès à chaque outil (dépôt GitHub, CVAT, MLflow, RustFS, Radicale, agenda de l'équipe), sans vous reconnecter. Les outils sont ouverts au fur et à mesure de la formation : une vignette grisée, avec un cadenas, n'est pas encore disponible.

Votre profil donne aussi le compte CalDAV de votre squad, partagé par ses membres : un agenda commun `calevent` et un agenda personnel pour vos tests. La vignette « Agenda de l'équipe » affiche leurs événements, chaque agenda dans sa couleur.



Bloqué ? Le bouton « Besoin d'aide ? » du tableau de bord prévient l'intervenant.



Ouvrez le portail, CVAT et MLflow avec la même adresse (`13.37.102.106`) : la connexion ne suit pas d'une adresse à l'autre.

## 4. Organisation du travail

Chaque squad travaille dans **son propre fork** du projet `https://github.com/CassiaFa/flexistaff` :

```
# 1. Forker le dépôt depuis l'interface GitHub (un fork par squad)
# 2. Cloner son fork
git clone https://github.com/<votre-compte>/flexistaff.git
cd flexistaff

# 3. Travailler conjointement dans ce fork, à vous de vous organiser (branche par personne ou par tâche)
```

Organisation conseillée (à adapter) :

```
flexistaff/
├── docs/                       # Documents du cas pratique (fournis par l'intervenant)
├── ressources/                 # Ressources complémentaires publiées en cours de formation
├── static/                     # Ressources statiques (images, logo…)
├── livrables/                  # Vos livrables (guide de labellisation, comptes rendus…)
│   └── guide_labellisation.md   # livrable J1
├── ai_training/                # Votre code d'entraînement (notebooks, scripts…)
│   └── train_yolo.ipynb         # code d'entraînement
├── dataset/                    # Votre dataset local (non versionné, voir .gitignore)
├── OCR/                        # Votre code OCR et parsing
├── .gitignore
└── README.md
```

Les modèles (`best.pt`, `.onnx`) ne sont pas dans le dépôt : ils sont rangés dans MLflow avec chaque run.



Committez souvent, avec des messages clairs. Ne committez jamais de mot de passe ni de gros fichiers binaires.

# 5. Jour 1 : labelliser, entraîner et tracer un premier modèle

## 5.1. Objectifs

- Définir **ensemble** ce que le modèle doit détecter, et l'écrire dans un guide.
- Labelliser environ **40 images par squad** (10 à 14 par personne).
- Entraîner un premier modèle YOLO, **tracé dans MLflow dès ce premier run**, et regarder ses résultats.

## 5.2. Labellisation avec CVAT

CVAT est un outil de labellisation open source, conçu pour la vision par ordinateur et utilisable en entreprise : plusieurs utilisateurs, plusieurs organisations, revue des annotations. Chaque utilisateur a un espace personnel et accède aussi aux espaces des organisations auxquelles il appartient.

Le travail s'organise ainsi :

```
Organisation --> Projets --> Tâches --> Jobs
```

- Le **projet** définit le type d'annotation (boîtes, masques, …) et ses **labels** (nom, couleur).
- La **tâche** regroupe un jeu d'images à annoter (envoyées depuis un poste ou lues dans un stockage cloud).
- Les **jobs** découpent la tâche en sous-ensembles, ce qui permet d'en assigner un à chaque personne.

Ce qui est déjà en place pour vous : le projet de votre squad, créé au choix de l'équipe sur le portail (sans labels : c'est le chef qui les crée, voir l'étape 1). Une fois toutes les équipes formées, l'intervenant crée la première tâche (`lot-1`) : les images à annoter, découpées en un job par membre (10 à 14 images chacun), chaque job étant assigné à son annotateur.

Pour annoter un nouveau lot d'images, déposez-le dans le dossier de votre squad sur RustFS (`cvat-data/<squad>/lot-2/`, par exemple `cvat-data/team-rocket/lot-2/`), puis créez la tâche depuis la carte « Nouvelle tâche d'annotation » du tableau de bord.

### 5.2.1. Se connecter

Ouvrez CVAT depuis le tableau de bord du portail : vous y êtes connecté automatiquement. L'adresse directe est `http://13.37.102.106:8080` ; si vous n'êtes pas connecté, elle vous renvoie vers le portail.



En haut à droite, choisissez l'organisation **FlexiStaff** (et non votre espace personnel), sinon vous ne verrez rien.

### 5.2.2. Étape 1 : décider ensemble avant d'annoter

Avant de se lancer, la squad décide :

1. **Quoi détecter ?** Quelles zones sont utiles pour reconstituer un créneau ? (la cellule d'un créneau, l'en-tête qui porte la date, l'étiquette d'horaire, …)
2. **Quelles classes ?** Trois classes claires valent mieux que huit floues.

3. **Quelles règles de tracé ?** Définir précisément les règles de tracé, afin de minimiser les écarts d'annotation entre annotateurs.
4. **Que faire des cas limites ?** Par exemple : une ligne « Cariste » est un poste, pas un horaire. L'annote-t-on ? Avec quelle classe ?

Le chef d'équipe crée ensuite les labels dans le projet. **L'ordre et les noms comptent** : l'ordre des labels donne les index de classe utilisés par YOLO.

### 5.2.3. Étape 2 : écrire le guide de labellisation

C'est un livrable. Une page suffit, mais elle doit permettre à **n'importe qui** d'annoter comme votre équipe :

- définition de chaque classe, avec un exemple ;
- règles de tracé (où commence et où s'arrête la boîte) ;
- liste des cas limites et de la décision prise.

Un exemple de guide de labellisation vous est fourni dans le dossier `ressources/` du dépôt Git, afin de vous aider à rédiger le vôtre.

### 5.2.4. Étape 3 : annoter, puis faire un contrôle croisé

1. Annotez votre job.
2. Échangez 5 images avec un coéquipier : chacun annote les mêmes, comparez. Les écarts révèlent les règles floues : corrigez le guide.
3. Revoyez vos annotations avant l'export.

| Raccourci CVAT | Action |
|---|---|
| `N` | Nouvelle forme (boîte) |
| `F` / `D` | Image suivante / précédente |
| `Ctrl` + `S` | Sauvegarder |
| `Suppr` | Supprimer la forme sélectionnée |



> Un modèle n'apprend que ce que vous lui montrez. Des boîtes décalées, de mauvaises classes ou des cellules oubliées lui apprennent l'hésitation : c'est le principe du *garbage in, garbage out*.

### 5.2.5. Étape 4 : exporter

Quand tout le monde a terminé, le **chef d'équipe** exporte la tâche entière : menu de la tâche > *Export task dataset* > format **Ultralytics YOLO Detection 1.0** (et non « YOLO 1.1 », l'ancien format Darknet). Vous obtenez une archive `.zip` déjà organisée pour Ultralytics : `data.yaml`, `images/` et `labels/`. Il la partage ensuite avec l'équipe.



> Seul le chef (assigné au projet) exporte la tâche entière. Un autre membre n'exporte que son job (menu du job > *Export job dataset*) : utile pour vérifier ses annotations.

Une annotation YOLO est un fichier `.txt` par image, une ligne par boîte :

```
# classe  cx     cy     w      h
0         0.431  0.317  0.120  0.089
1         0.431  0.205  0.120  0.051
```

`cx`, `cy` = centre de la boîte ; `w`, `h` = largeur et hauteur. Toutes les valeurs sont **normalisées** entre 0 et 1 (par rapport à la taille de l'image).

## 5.3. Préparer le jeu de données

Ultralytics attend cette structure. Vous devrez **découper** vos images en deux ensembles : `train` (le modèle apprend) et `val` (on mesure sans que le modèle l'ait vu).

```
dataset/
├── data.yaml
├── images/
│   ├── train/   *.jpg
│   └── val/     *.jpg
└── labels/
    ├── train/   *.txt     (même nom que l'image)
    └── val/     *.txt
```

Fichier `data.yaml` (adaptez les classes aux vôtres, dans l'ordre de CVAT) :

```yaml
path: /content/dataset        # à adapter à votre environnement
train: images/train
val: images/val

nc: 3                         # nombre de classes
names:
  0: nom_classe_0
  1: nom_classe_1
  2: nom_classe_2
```



Ordre de grandeur : environ 80 % des images en `train`, 20 % en `val`. Avec si peu d'images, les scores varieront beaucoup d'un tirage à l'autre : ne sur-interprétez pas un écart de quelques points.

## 5.4. Tracer avec MLflow dès le premier entraînement

Tous vos entraînements sont tracés dans MLflow, **dès le premier**. Sans suivi, trois semaines plus tard personne ne sait quel modèle est en production, avec quelles données ni quels paramètres.

| Brique | Rôle | En formation |
|---|---|---|
| Tracking | Chaque run : paramètres, métriques, fichiers | ✔ |
| Models | Format standard pour empaqueter un modèle | |
| Registry | Catalogue versionné de modèles, avec alias (« champion ») | ✔ (jour 2) |

Vocabulaire : une **expérience** regroupe des **runs** ; un run = un entraînement. Interface web : `http://13.37.102.106:5000`

### 5.4.1. Préparer le notebook

Dans Colab, activez d'abord le GPU : *Exécution* > *Modifier le type d'exécution* > *GPU (T4)*. Changer de type d'exécution réinitialise la session : faites-le **avant** d'installer quoi que ce soit.

```
pip install ultralytics mlflow onnx onnxruntime
```

### 5.4.2. Se connecter à MLflow

MLflow est protégé par le portail : le client MLflow doit présenter vos identifiants (les mêmes que pour CVAT). Vous n'avez **pas** besoin des identifiants S3 : les artefacts transitent par le serveur MLflow.

**Méthode recommandée : le fichier d'identifiants.** Sur votre profil du portail, téléchargez `identifiants-<login>.json`, puis déposez-le dans Colab (panneau *Fichiers*, à gauche).

```
import json, os
import mlflow

creds = json.load(open("identifiants-<login>.json"))["mlflow"]
os.environ["MLFLOW_TRACKING_USERNAME"] = creds["username"]
os.environ["MLFLOW_TRACKING_PASSWORD"] = creds["password"]

mlflow.set_tracking_uri(creds["tracking_uri"])
mlflow.set_experiment("flexistaff_<squad>")
```

**Sans le fichier**, saisissez le mot de passe au clavier plutôt que de l'écrire dans le notebook :

```
import os
from getpass import getpass
import mlflow

os.environ["MLFLOW_TRACKING_USERNAME"] = "<login>"
os.environ["MLFLOW_TRACKING_PASSWORD"] = getpass("Mot de passe : ")

mlflow.set_tracking_uri("http://13.37.102.106:5000")
mlflow.set_experiment("flexistaff_<squad>")
```

`<squad>` est le nom de votre équipe en minuscules, espaces remplacés par des tirets : `team-rocket`, `team-magma` ou `team-aqua`.



Le fichier d'identifiants contient vos mots de passe : ne le committez jamais et ne le laissez pas dans un notebook partagé.

| Élément d'un run | Ce qu'on y met |
|---|---|
| params | ce qu'on a **choisi** (modèle, epochs, taille d'image, version du dataset) |
| metrics | ce qu'on a **obtenu** (mAP50, précision, rappel) |
| artifacts | ce qu'on a **produit** (`best.pt`, courbes) |
| tags | squad, version du dataset |

Convention de nommage : `run_name = "<membre>_<version>"`. Le premier entraînement de Toto s'appelle donc `toto_v1`, les suivants `toto_v2`, `toto_v3`… Tous les membres de la squad tracent leurs runs dans l'expérience commune `flexistaff_<squad>`.

## 5.5. Premier entraînement YOLO

YOLO (*You Only Look Once*) détecte tous les objets en une seule passe. Notre tâche est de la **détection** : on localise chaque zone (boîte + classe + confiance), on lira ensuite son contenu.

```
import onnx
import mlflow
from ultralytics import YOLO, settings

settings.update({"mlflow": False})  # suivi MLflow fait à la main ci-dessous (sinon conflit de paramètres)
model = YOLO("yolov8s.pt")          # poids pré-entraînés : on ne repart pas de zéro

with mlflow.start_run(run_name="toto_v1"):                    # <membre>_<version>
    mlflow.set_tags({"squad": "team-rocket", "dataset": "v1"})  # à adapter
    mlflow.log_params({"model": "yolov8s", "epochs": 50, "imgsz": 640})

    res = model.train(data="data.yaml", epochs=50, imgsz=640)
    mlflow.log_metrics({"mAP50": res.box.map50,      # métriques sur val
                        "precision": res.box.mp,
                        "recall": res.box.mr})       # celle qui compte le plus

    best_pt = model.trainer.best    # runs/detect/train*/weights/best.pt
    mlflow.log_artifact(str(best_pt))

    # Export ONNX, rangé dans le run : c'est le format que charge l'application
    onnx_path = YOLO(best_pt).export(format="onnx", imgsz=640)
    mlflow.onnx.log_model(onnx.load(onnx_path), name="onnx",
                          save_as_external_data=False)
```

Chaque run contient ainsi les poids `best.pt` (onglet *Artifacts*) et leur version ONNX (onglet *Logged models*). ONNX est un format d'échange indépendant du framework : c'est lui que l'application chargera au jour 3.

### Pourquoi exporter en ONNX ?

`best.pt` sert à **entraîner** ; ONNX sert à **déployer**. Un modèle en production doit être léger, portable et indépendant de l'outil qui l'a produit.

| Atout | Concrètement |
|---|---|
| Léger | Pour l'utiliser, `onnxruntime` (quelques dizaines de Mo) suffit : ni PyTorch ni Ultralytics (plusieurs Go). L'image Docker et la mémoire du service en dépendent. |
| Portable | Le même fichier tourne sur CPU, GPU, téléphone, et depuis d'autres langages que Python. |
| Découplé | L'application ne dépend pas de la version d'Ultralytics utilisée pour entraîner ; les noms de vos classes sont rangés dans le fichier (métadonnées). |
| Rapide | Inférence souvent plus rapide sur CPU, sans GPU côté serveur. |

Les contreparties : la taille d'image est fixée à l'export (`imgsz=640`), le post-traitement (seuil de confiance, suppression des boîtes en double) est à refaire côté application, et les versions d'`onnx` / `onnxruntime` doivent rester compatibles.



La ligne de commande `yolo detect train …` fonctionne aussi, mais ne trace rien dans MLflow : pour la formation, utilisez le code Python ci-dessus.

Les résultats sont dans `runs/detect/train*/`, un nouveau dossier à chaque entraînement (`train`, `train-2`, `train-3`…) : poids dans `weights/best.pt`, courbes, matrice de confusion, `results.csv`.

| Taille | Usage |
|---|---|
| `n` (nano) | Le plus rapide : bon pour démarrer |
| `s` (small) | Bon compromis : notre choix par défaut |
| `m`, `l`, `x` | Plus précis, plus lents : inutiles avec si peu de données |

Regarder une prédiction à l'œil reste le meilleur premier test :

```
yolo detect predict model=runs/detect/train/weights/best.pt source=photo.jpg conf=0.25 save=True
```

Adaptez le dossier (`train-2`, `train-3`…) au run que vous voulez tester.

## 5.6. Enregistrer votre premier champion (Registry)

Chaque membre a maintenant son run `<membre>_v1`, avec son modèle ONNX (code ci-dessus). Comparez-les dans MLflow (sélectionnez les runs puis *Compare*) et choisissez le meilleur avec le critère de promotion de la formation :

1. le **rappel** d'abord : une case oubliée, c'est un créneau perdu ;
2. le **mAP50** pour départager deux modèles proches ;
3. c'est le **chef** de squad qui décide.

Le chef enregistre ensuite ce modèle dans le registre, sous le nom de la squad, avec l'alias « champion » : c'est la version que chargent le notebook de test OCR (jour 2) et l'application (jour 3).

```
import mlflow
from mlflow import MlflowClient

RUN_ID = "<run_id>"              # identifiant du run choisi (page du run dans MLflow)
NAME = "flexistaff_<squad>"      # ex. flexistaff_team-rocket

mv = mlflow.register_model(f"runs:/{RUN_ID}/onnx", NAME)
MlflowClient().set_registered_model_alias(NAME, "champion", mv.version)
```



N'utilisez pas `with mlflow.start_run(run_id=…)` pour compléter un run terminé : MLflow le rouvre et réécrit son heure de fin, la durée du run devient fausse. L'enregistrement ci-dessus ne modifie pas le run.

Pour récupérer le fichier ONNX en local :

```
mlflow.artifacts.download_artifacts(f"models:/{NAME}@champion", dst_path="champion")
# -> champion/model.onnx
```

## 5.7. Livrables du jour 1

| Livrable | Détail | Où |
|---|---|---|
| Guide de labellisation | Markdown (ou autre format lisible) | `livrables/` |

| Livrable | Détail | Où |
|---|---|---|
| Jeu de données labellisé | Export CVAT, découpé en `train` / `val` | CVAT |
| Code d'entraînement | Notebook ou script qui relance l'entraînement | `ai_training/` |
| Premier run MLflow | `<membre>_v1` (ex. `toto_v1`) visible dans MLflow, avec paramètres, métrique, `best.pt` et modèle ONNX | MLflow |
| Premier modèle | `best.pt` et une capture de la prédiction sur une photo | MLflow |
| Premier champion | `flexistaff_<squad>` version 1 dans le registre, alias « champion » | MLflow |

## 5.8. Checklist de fin de journée

- ☐ Le guide de labellisation est écrit et relu par toute la squad
- ☐ Toutes les images de la squad sont annotées
- ☐ Le dataset est exporté et découpé en `train` / `val`
- ☐ Un entraînement s'est terminé sans erreur
- ☐ Ce run apparaît dans MLflow (paramètres, métrique, `best.pt`, modèle ONNX, tags squad et dataset)
- ☐ Vous avez regardé une prédiction sur une vraie photo
- ☐ Le meilleur `v1` de la squad est enregistré avec l'alias « champion »
- ☐ Le travail est poussé sur le fork de la squad

# 6. Jour 2 : évaluer, améliorer, comparer, promouvoir

## 6.1. Objectifs

- Continuer à tracer **chaque** entraînement dans MLflow (plus de `best_v2_final_OK.pt`).
- Comprendre les métriques et décider de ce qu'il faut améliorer.
- Entraîner des challengers, les comparer au champion et promouvoir le meilleur.
- Tester la reconnaissance OCR sur les détections du modèle entraîné, afin de valider et commencer à réfléchir à la structuration (parsing).

## 6.2. Partir du run de la veille

Ouvrez MLflow (`http://13.37.102.106:5000`) et retrouvez le **champion** de votre squad (onglet *Models*, `flexistaff_<squad>`) : c'est votre **référence**. Chaque amélioration sera un nouveau run (`v2`, `v3`, …), avec le même code de suivi qu'au jour 1 (mAP50, précision et rappel).

## 6.3. Tester l'OCR sur le champion

Avant d'améliorer le modèle, regardez ce que l'OCR lit dans les zones qu'il détecte. Ouvrez le notebook `ressources/test_ocr_champion.ipynb` (Colab ou local), renseignez votre fichier d'identifiants et votre squad : il télécharge le modèle `flexistaff_<squad>@champion` et le jeu d'images de test commun (photos de plannings jamais vues à l'entraînement, les mêmes pour toutes les squads), détecte les zones, puis passe chacune à EasyOCR (mêmes réglages que l'application du jour 3). Il accepte aussi vos images (`IMAGES`).

Vous obtenez, image par image, le texte lu dans chaque boîte, puis un bilan par classe. Servez-vous-en pour :

- repérer les classes dont le texte est illisible ou vide (boîte trop serrée, trop grande, mal définie dans le guide de labellisation ?) ;
- commencer à réfléchir au **parsing** : quelle classe porte la date, l'heure, le titre ? comment rattacher un événement à son jour ? quels formats d'heure rencontrez-vous ?

Le même notebook sert plus loin à tester un challenger (`ALIAS = "challenger"`).

## 6.4. Évaluer son modèle

| Métrique | Définition | Si trop bas |
|---|---|---|
| IoU | Intersection ÷ union entre la boîte réelle et la boîte prédite. IoU ≥ 0,5 : détection comptée juste. | - |
| Précision | Parmi mes détections, combien sont justes ? | fausses alarmes |
| Rappel | Parmi les vrais objets, combien ai-je trouvés ? | oublis |
| mAP50 | Précision moyenne avec IoU ≥ 0,5 : départage deux modèles de rappel proche | - |



Pour notre cas, le **rappel sur la classe « cellule de créneau »** compte le plus : une cellule oubliée, c'est un créneau perdu.

```
yolo detect val model=best.pt data=data.yaml
```

Regardez aussi la **matrice de confusion** (`runs/detect/train*/confusion_matrix.png`) : elle montre quelles classes se confondent ou quelles zones sont oubliées.

## 6.5. Améliorer

Une seule modification à la fois, un run MLflow par modification, sinon vous ne saurez pas ce qui a marché :

- **Les données d'abord** : corriger des annotations incohérentes, ajouter des images difficiles. Un mauvais score vient presque toujours des données.
- **L'augmentation** : rotation, luminosité, flou (une image devient dix).
- **Les paramètres** : nombre d'epochs, taille d'image, taille de modèle.

Pièges classiques :

- **Objets denses** : 30 à 50 cellules collées, la suppression des doublons (NMS) peut fusionner des voisines.
- **Surapprentissage** : très bon score sur `train`, mauvais sur `val`.
- **Synthétique ≠ réel** : testez toujours sur de vraies photos.

## 6.6. Ajouter des données simulées

Vos 40 images annotées, c'est peu. Pour en ajouter sans annoter, un **générateur** dessine des plannings (grilles mensuelles, plannings hebdomadaires, plannings à codes) dont il connaît exactement les boîtes. Il est fourni dans `ressources/generateur/`, avec un fichier de labels préparé pour votre squad à partir de votre guide de labellisation : `ressources/generateur/labels/<squad>.yaml`.

Récupérez d'abord la mise à jour du dépôt de l'intervenant (voir « Récupérer les mises à jour » dans le README), puis, dans Colab, ajoutez les images générées à votre export CVAT :

```
!python ressources/generateur/generate_calendar_custom.py \
    --labels ressources/generateur/labels/<squad>.yaml \
    --n 200 --prefix synth --mode merge --out dataset
```

- `--mode merge` ajoute les images à votre dataset (dossier de l'export, avec son `data.yaml`) ; les nouvelles s'appellent `synth_0000.jpg`, `synth_0001.jpg`… et aucune image existante n'est écrasée.
- Il vérifie vos classes (**même ordre** que le `data.yaml`, sinon arrêt) ; 20 % des images vont en `val`.

Tracez ce nouveau jeu dans MLflow : le tag `dataset` passe à `v2`, et le nombre d'images de chaque origine est enregistré en paramètre.

```
from pathlib import Path

imgs = list(Path("dataset/images/train").glob("*.jpg"))
n_synth = sum(p.name.startswith("synth_") for p in imgs)
mlflow.log_params({"n_images_cvat": len(imgs) - n_synth, "n_images_synth": n_synth})
mlflow.set_tag("dataset", "v2")    # v1 : images CVAT seules ; v2 : + images simulées
```



Le simulé ne remplace pas le réel : ses images restent plus « propres » que vos photos. N'entraînez jamais sur le simulé seul, restez autour de 300 images ajoutées au maximum, et jugez le résultat sur le **jeu de test commun** (notebook de test OCR et métriques), pas seulement sur `val` (qui contient du simulé).



Une bonne expérience de la journée : comparer un challenger `v2` au champion `v1`, à paramètres d'entraînement identiques. Si le rappel ne progresse pas sur les vraies photos, le simulé n'aide pas (encore) : essayez une autre quantité (`--n 100`, `--n 300`) ou demandez à faire évoluer votre fichier de labels.

## 6.7. Comparer les runs

Dans l'interface MLflow, sélectionnez plusieurs runs puis *Compare*.



La bonne question n'est pas « qui a gagné ? » mais « qu'est-ce qui a changé entre `toto_v1` et `toto_v2`, et pourquoi ça a marché ? ».

## 6.8. Challenger et champion

Le champion reste en place tant qu'un modèle ne fait pas mieux. Quand un run vous semble meilleur, enregistrez-le comme **challenger** :

```
import mlflow
from mlflow import MlflowClient

RUN_ID = "<run_id>"              # identifiant du run candidat
NAME = "flexistaff_<squad>"      # ex. flexistaff_team-rocket
client = MlflowClient()

mv = mlflow.register_model(f"runs:/{RUN_ID}/onnx", NAME)
client.set_registered_model_alias(NAME, "challenger", mv.version)
```

Comparez-le au champion : métriques dans MLflow, puis rendu OCR (notebook de test, `ALIAS = "challenger"`).



Critère de promotion : le **rappel** d'abord, le **mAP50** pour départager, un coup d'œil au rendu OCR. C'est le **chef** de squad qui décide.

Si le challenger l'emporte, le chef le promeut :

```
v = client.get_model_version_by_alias(NAME, "challenger").version
client.set_registered_model_alias(NAME, "champion", v)     # l'alias quitte l'ancienne version
client.delete_registered_model_alias(NAME, "challenger")
```

Sinon, gardez le champion et notez pourquoi : c'est aussi un résultat.

## 6.9. Bonus : et sans IA ?

Faut-il toujours entraîner un modèle ? Un planning imprimé avec des bordures est une **grille** : des traits horizontaux, des traits verticaux, des cases. Le traitement d'image classique (OpenCV) la retrouve sans annotation, sans entraînement et sans GPU. Le notebook `ressources/grille_sans_ia.ipynb` la construit pas à pas et mesure son rappel ; CalEvent en embarque une version complète. Le but n'est pas de remplacer YOLO, mais de savoir *quand* une approche classique suffit.

**La chaîne de traitement.** Elle produit les mêmes rôles que YOLO (`event`, `date_cell`, `resource_header`, `month_header`, `day_header_row`, `calendar`) : l'OCR et le parser du jour 3 tournent derrière sans changement.

| Étape | Opération OpenCV | Résultat |
|---|---|---|
| 1. Image de travail | Réduction (côté max 1600 px), canal « valeur » `max(B, G, R)` plutôt que niveaux de gris : un trait gris reste plus sombre qu'une case pastel | Image légère, traits contrastés |
| 2. Binarisation | Seuillage adaptatif (`cv2.adaptiveThreshold`) : encre = nettement plus sombre que la moyenne locale | Image noir et blanc, insensible aux ombres |

| Étape | Opération OpenCV | Résultat |
|---|---|---|
| 3. Traits | Morphologie : ouverture avec un noyau long et fin, horizontal puis vertical (`cv2.morphologyEx`) ; le texte, trop court, disparaît | Masque des traits H et masque des traits V |
| 4. Tableau | Intersections des deux masques (`cv2.bitwise_and`) ; on garde l'amas le plus peuplé (le bord de la feuille n'en a que 4), enveloppe convexe puis `approxPolyDP` | Quadrilatère du tableau |
| 5. Redressement | `cv2.getPerspectiveTransform` + `cv2.warpPerspective` ; si la colonne des noms n'est pas à gauche, rotation d'un quart ou d'un demi-tour | Tableau vu de face |
| 6. Grille | Projection des traits : part du tableau traversée par chaque ligne et colonne, pics = lignes de la grille ; traits invisibles rétablis si la grille est régulière | Listes de lignes et de colonnes |
| 7. Cases | Chaque paire de lignes × paire de colonnes délimite une case ; les lignes sans traits verticaux en haut forment le titre | Une boîte par case |
| 8. Test d'encre | Part de pixels d'encre au centre de la case, comparée à un seuil (environ 1,2 %) | Case vide ou non |
| 9. Rôles | Selon la *place* : 1re ligne = `date_cell`, 1re colonne = `resource_header`, case encrée du corps = `event`, titre = `month_header`, tableau = `calendar`. Retour aux coordonnées de la photo d'origine par l'homographie inverse | Mêmes détections que YOLO |

Ce détecteur est donc un *système de règles* : aucune valeur n'est apprise, chaque seuil est écrit à la main et se règle en regardant des images. La confiance est une heuristique (régularité de la grille, part des intersections trouvées), pas une probabilité : une grille nette donne environ 0,9, une grille douteuse 0,5 à 0,8 (« à revoir »).

**Ce que ça donne.** Mesuré sur 10 plannings d'équipe (rosters) pris de trois façons : à plat, photo simulée (perspective, rotation de ±3°, ombre, bruit, compression JPEG) et tourné d'un quart de tour, soit 30 images.

| Jeu de test | Rappel | Précision |
|---|---|---|
| 30 images de rosters (total) | 0,995 | 1,000 |
| Rosters à plat | 1,00 | - |
| Photos simulées | 0,93 à 1,00 | - |
| Plannings synthétiques sans grille nette (générateur du banc d'essai) | 0,45 à 0,55 | - |

Les rares cases ratées se trouvent au bord droit du tableau, là où la perspective écrase les traits. À l'inverse, dès que la grille n'est plus nette, le rappel s'effondre : l'approche excelle dans son domaine, échoue en dehors.

| Avantages | Limites |
|---|---|
| • Aucune annotation, aucun entraînement<br>• Pas de GPU, très rapide, déploiement léger<br>• Déterministe : même image, même résultat<br>• Explicable : on sait pourquoi une case est détectée ou manquée | • Il faut des bordures visibles (export Excel sans quadrillage : échec)<br>• Calendriers mensuels refusés (7 colonnes égales, hors périmètre)<br>• Forte inclinaison : échec à 25°, traits très fins perdus<br>• Chaque nouveau format demande de nouvelles règles codées à la main |

**Alors, YOLO ou OpenCV ?** YOLO coûte de l'annotation et de l'entraînement, mais il *généralise* : photos tordues, cases colorées, formats que vous n'avez pas prévus, pourvu que le jeu d'entraînement les ait vus. OpenCV ne coûte rien à mettre en route, mais ne vaut que par ses hypothèses.



Le critère de choix reste le même que pour la promotion d'un modèle : le **rappel d'abord** (une case oubliée est un créneau perdu), puis la **variété des formats** à couvrir. Un seul format net et bordé : l'approche classique suffit, et elle est plus simple à défendre. Des formats multiples ou des photos difficiles : un modèle appris, éventuellement avec l'approche classique en point de comparaison (*baseline*) : si votre YOLO ne fait pas mieux qu'elle, c'est un signal.

### 6.10. Livrables du jour 2

| Livrable | Détail |
|---|---|
| Test OCR | Le notebook de test a tourné sur le champion ; vos remarques sur le texte lu par classe |
| Au moins un challenger | Enregistré dans `flexistaff_<squad>` et comparé au champion (métriques et OCR) |
| Note de synthèse | 5 lignes : ce que vous avez changé, ce que ça a donné, la décision de promotion et sa justification |

### 6.11. Checklist de fin de journée

- ☐ Tous vos entraînements du jour sont dans MLflow, avec squad et version du dataset
- ☐ Vous savez expliquer la différence entre précision et rappel
- ☐ Le notebook de test OCR a tourné sur le champion
- ☐ Au moins un challenger comparé au champion, décision notée
- ☐ L'alias « champion » désigne le meilleur modèle
- ☐ Le modèle « champion » se télécharge (`models:/flexistaff_<squad>@champion`)

## 7. Jour 3 : du texte lu à l'agenda

### 7.1. Objectifs

- Transformer le texte lu par l'OCR en événements structurés (**parsing**), avec une confiance.
- Publier ces événements dans l'agenda de votre squad (CalDAV).
- Voir votre champion tourner dans l'application de démonstration **CalEvent**.
- Présenter une démonstration de bout en bout.

```
Photo -> YOLO (champion) -> EasyOCR -> ocr_champion.json -> VOTRE PARSER -> événements -> CalDAV (Radicale)
         |------------ fait au jour 2 ------------|        |--- jour 3 ---|            |-- fourni --|
```

Le cœur de la journée est le **parsing** : c'est là que se joue la qualité du pipeline, et il dépend de **vos** classes. La publication CalDAV vous est fournie, prête à l'emploi.

## 7.2. Point de départ

- Le fichier `ocr_champion.json` produit au jour 2 par `ressources/test_ocr_champion.ipynb` (relancez le notebook si besoin, éventuellement sur vos propres photos avec le paramètre `IMAGES`).
- Le notebook `ressources/parsing_jour3.ipynb` : il charge ce JSON, avec les fonctions à écrire et des cas de test.
- Le module `ressources/caldav_publier.py` : la fonction `publier(…)` qui envoie vos événements dans Radicale.
- Votre fichier d'identifiants (profil du portail) : le compte CalDAV de la squad est sous la clé `caldav`.

## 7.3. Parsing défensif

L'OCR se trompe. Exemple : `Mar . 1Z/03 I4h-18h` doit devenir `12/03 14:00 → 18:00` avec une confiance de 0,62.

| Principe | Détail |
|---|---|
| Corriger | O→0, I→1, Z→2 **uniquement** dans les zones de chiffres |
| Tolérer | `14h-18h`, `14:00-18:00`, `de 14h à 18h`, `9h30` |
| Croiser | La date vient de la case ou de la colonne (en-tête de date, en-tête du mois), pas seulement de la cellule lue |
| Noter | Chaque correction fait baisser la confiance |

Pour rattacher un événement à sa date, servez-vous de la **géométrie** : chaque zone du JSON a sa boîte. Un créneau appartient à la case qui le contient, ou à la colonne sous l'en-tête de date. Vos classes sont les vôtres : adaptez le parser à **votre** guide de labellisation.

Schéma de sortie visé, pour chaque événement :

```
{
  "title": "M. Bernard - Entrepôt Nord",
  "date": "2026-03-12",
  "start_time": "14:00",
  "end_time": "18:00",
  "location": null,
  "confidence": 0.62
}
```

- Date en ISO 8601 (`YYYY-MM-DD`), heures sur 24 h (`HH:MM`).
- `start_time`, `end_time` et `location` valent `null` s'ils sont absents (sans heure de début : journée entière).
- Champs optionnels pour les plannings à codes : `person` (la personne), `all_day` (journée entière) et `overnight` (poste de nuit, fin le lendemain).

### 7.3.1. Plannings à codes

Dans un planning d'équipe (personnes en lignes, jours en colonnes), chaque case porte un code (`M06`, `N08`, `MAL`…) :

- la classe `agent` (la personne) se rattache au rôle `resource_header` via `CLASS_MAP` ; la personne se trouve par la **ligne**, la date par la **colonne** ;
- une **table des codes** (`CODES`) : repos = journée entière, nuit (`N08`) jusqu'au lendemain ;

- `MAL` devient « Absence » : un arrêt maladie est une donnée de santé, jamais dans un agenda partagé (RGPD) ;
- en-tête sur deux mois (« Novembre / Décembre ») : les jours qui repartent à 1 sont ceux du mois suivant.



Écrivez vos cas de test **avant** de coder : une ligne de texte OCR bruité, le résultat attendu. Le notebook en propose quelques-uns, ajoutez ceux que vous rencontrez sur vos photos.

### 7.3.2. L'humain dans la boucle

| Confiance | Statut | Comportement |
|---|---|---|
| ≥ 0,8 | Pré-coché | un tap pour valider |
| 0,5 - 0,8 | À vérifier | mis en évidence |
| < 0,5 | Non proposé | saisie manuelle |



Les seuils sont des choix de produit. Vous devrez justifier les vôtres. Rappelez-vous que le **rappel** compte : un créneau manqué est un créneau perdu.

## 7.4. Publier dans l'agenda (CalDAV)

La publication est fournie : `ressources/caldav_publier.py`. Elle respecte les trois règles d'un bon client CalDAV :

1. **UID stable** : calculé à partir de la date, des heures et du titre ; le même créneau a toujours le même UID.
2. **Fuseau explicite** : `TZID=Europe/Paris`, sinon décalages d'une ou deux heures.
3. **Anti-doublon** : si l'UID est déjà dans l'agenda, l'événement n'est pas recréé (statut « déjà présent »).

```
from caldav_publier import publier, charger_identifiants

ident = charger_identifiants("identifiants-<login>.json")  # clé « caldav » : compte de la squad
resultats = publier(evenements, **ident, dry_run=True) # simulation : rien n'est envoyé
resultats = publier(evenements, **ident)               # envoi dans l'agenda « calevent »
```

Chaque événement renvoie un statut : `créé`, `déjà présent` ou `erreur: …`. Publiez d'abord en simulation, ne publiez que les événements validés (seuils de confiance), puis vérifiez sur la vignette « Agenda de l'équipe » du portail (`http://13.37.102.106/agenda`) : après « Relire l'agenda », ils apparaissent à la couleur de leur agenda, marqués « Nouveau » (l'interface `/.web/` de Radicale ne les montre pas).

Un événement iCalendar tel que l'écrit `publier` :

```
BEGIN:VEVENT
UID:3f1c9e1a-...@calevent
DTSTART;TZID=Europe/Paris:20260312T140000
DTEND;TZID=Europe/Paris:20260312T180000
SUMMARY:M. Bernard - Entrepôt Nord
END:VEVENT
```



Lisez le code de `publier` : pourquoi un UID stable, et si l'OCR lit le titre autrement ?



*Facultatif : voir l'agenda sur votre PC ou votre téléphone*

Abonnez un client CalDAV au compte Radicale de la squad (identifiants sur votre profil), adresse `http://13.37.102.106:5232/`. Serveur en HTTP, mot de passe en clair : supprimez ce compte de l'appareil après la formation.

- **PC** : Thunderbird, Nouvel agenda > Sur le réseau, puis l'adresse et les identifiants.
- **Android** : DAVx⁵ (gratuit sur F-Droid, payant sur le Play Store), qui synchronise avec l'agenda du téléphone.
- **iPhone** : Réglages > Calendrier > Comptes > Ajouter un compte > Autre > Compte CalDAV (désactivez SSL dans les réglages avancés).

## 7.5. L'application de démonstration : CalEvent

**CalEvent** enchaîne tout le pipeline dans une application : photo → détection → OCR → parsing → validation → agenda. Une instance tourne sur la VM ; l'intervenant ouvre son accès depuis le portail.

| | |
|---|---|
| Interface web | `http://13.37.102.106:8000/` |
| Version mobile (dans le navigateur du téléphone) | `http://13.37.102.106:8000/app` |
| Comparaison multi-modèles | `http://13.37.102.106:8000/compare-view` |
| Documentation de l'API (Swagger) | `http://13.37.102.106:8000/docs` |

- Elle charge directement depuis MLflow le modèle `champion` de chaque squad (nom `<squad>@champion`) ; le bouton ⟳ recharge les modèles après une promotion.
- Ses événements partent dans les agendas de votre squad : vérifiez-les dans « Agenda de l'équipe » du portail.
- Le sélecteur de modèle propose aussi `grille-opencv (sans IA)`, le détecteur classique du bonus du jour 2 : comparez-le à votre champion dans `/compare-view`.
- Son parser connaît les classes du modèle de référence (`month_header`, `date_cell`, `event`…). Pour qu'il construise des événements avec **votre** modèle, donnez à l'intervenant la correspondance entre vos classes et ces rôles, par exemple `jour=date_cell, creneau=event, mois=month_header` (planning à codes : `agent=resource_header`).
- La démo propose aussi un sélecteur « Personnes », la case « Journée entière », le choix de l'agenda par événement (« Envoyer dans » : votre agenda personnel par défaut) et un bouton « Prendre une photo ».

Comparez ses résultats aux vôtres sur les mêmes photos : c'est une bonne source d'idées pour votre parser, et inversement.

## 7.6. Bonus recommandé : mettre votre champion en production

Jusqu'ici, votre modèle vivait dans un notebook. Un modèle *en production* est un **service avec un contrat** : n'importe quel programme (CalEvent, une appli mobile, le script d'un collègue) envoie une image en HTTP et reçoit des détections en JSON, sans rien savoir de YOLO. Vous exposez ici **votre** modèle `flexistaff_<squad>@champion` dans une petite API FastAPI, sans PyTorch ni Ultralytics (`onnxruntime`).

Le dossier `ressources/api_modele/` contient :

- `detecteur.py` (fourni) : inférence ONNX et téléchargement du modèle depuis le registre MLflow ;
- `api.py` (à compléter, 6 TODO) : le contrat de l'API est fourni, vous écrivez le corps des routes ;
- `mesurer_latence.py`, `requirements.txt`, `Dockerfile` : mesure, dépendances, image Docker.

```
cp -r ressources/api_modele livrables/   # on ne modifie pas ressources/
cd livrables/api_modele
pip install -r requirements.txt
export IDENTIFIANTS_JSON=identifiants-<login>.json    # MLflow : http://13.37.102.106:5000
export SQUAD=team-rocket
uvicorn api:app --port 8001       # puis http://localhost:8001/docs
```

À réaliser dans l'ordre (environ 1 h 30, à répartir dans la squad) :

1. `GET /health` : version du modèle chargé et **vos** classes ;
2. `POST /predict` : une image en entrée, les détections (classe, confiance, boîte) en JSON ;
3. mesurer la latence : `python mesurer_latence.py image.jpg -n 50` (moyenne et p95) ;
4. `POST /reload` : déplacez l'alias `champion` (promotion), rechargez, vérifiez que la version a changé **sans redémarrer** ; puis revenez en arrière en remettant l'alias ;
5. facultatif : `POST /parse`, qui appelle votre parser du jour 3 (`mon_parser.py`) ;
6. facultatif : `docker build` et `docker run`, en passant les identifiants avec `-e` (jamais dans l'image).



L'alias `champion` est partagé avec CalEvent et vos camarades. Prévenez la squad avant de le déplacer et remettez-le à sa place. Pour comparer sans risque, lancez un second service avec `ALIAS=challenger` sur un autre port.

Dans votre rapport du jour 3, donnez des **chiffres** : latence moyenne et p95, mémoire du service, taille de l'image Docker (ou de l'environnement), et pourquoi ONNX (dépendances, taille, portabilité, fichier unique versionné dans le registre).

## 7.7. Répartition du travail

Exemple pour une squad de 3 ou 4 :

| Sujet | Mission |
|---|---|
| Dates | Extraire la date d'une case ou d'une colonne (en-tête du mois, numéro du jour), rattacher chaque créneau à sa date |
| Heures et texte | Corriger le bruit OCR, reconnaître les formats d'heure, séparer heures et titre |
| Confiance et publication | Calculer la confiance, appliquer les seuils, publier avec `publier` et vérifier dans « Agenda de l'équipe » |
| Tests ou bonus | Écrire les cas de test sur vos photos, ou mettre le champion en production (API) |

## 7.8. Livrables du jour 3

- Votre parser : le notebook `parsing_jour3.ipynb` complété, ses cas de test au vert.
- Les événements d'au moins une photo publiés dans l'agenda `calevent`, sans doublon, à la bonne heure.
- Une **démonstration** sur une vraie photo de planning.
- Un court retour : ce qui marche, ce qui ne marche pas, vos seuils de confiance et pourquoi.

## 7.9. Checklist de fin de journée

- ☐ Le parser produit des événements au bon schéma, avec une confiance
- ☐ Les cas de test du notebook passent, y compris les vôtres
- ☐ Les événements apparaissent dans « Agenda de l'équipe », sans doublon, à la bonne heure
- ☐ La correspondance de vos classes est transmise à l'intervenant et votre champion tourne dans CalEvent
- ☐ Vous êtes prêts à présenter la démonstration

# 8. Dépannage (FAQ)

1. *CVAT : je ne vois aucune tâche, aucun job.*

   Organisation **FlexiStaff** (en haut à droite) ; la tâche n'apparaît qu'une fois les équipes formées.
2. *Mes boîtes ne sont pas cohérentes avec celles de mes coéquipiers.*

   Relisez et précisez le guide de labellisation, en particulier les règles de tracé. Annotez une image ensemble pour faciliter votre mise en accord.
3. *MLflow : « Connection refused » ou timeout depuis Colab.*

   Vérifiez l'adresse et le port (`curl http://13.37.102.106:5000/health`), sinon prévenez l'intervenant.
4. *MLflow : l'envoi de `best.pt` échoue.*

   Vérifiez `mlflow.set_tracking_uri(…)` : les artefacts passent par le serveur MLflow.
5. *Colab : « CUDA out of memory ».*

   Réduisez `batch` (par exemple `batch=8`) ou `imgsz`, ou prenez un modèle plus petit (`n`).
6. *YOLO ne trouve pas mes images ou mes labels.*

   Vérifiez les chemins du `data.yaml` et que `images/train/x.jpg` a bien son `labels/train/x.txt`.
7. *Mon score est très bon sur `train` et mauvais sur `val`.*

   Surapprentissage : plus de données, plus d'augmentation, moins d'epochs.
8. *OCR : le premier lancement est très long.*

   Normal : EasyOCR télécharge ses modèles une seule fois.
9. *CalDAV : erreur 401 (Unauthorized).*

   Mauvais identifiants. Testez-les sur l'interface web de Radicale.
10. *Les événements ont 1 h ou 2 h de décalage.*

    Fuseau manquant : utilisez `TZID=Europe/Paris` dans `DTSTART` et `DTEND`.
11. *Événements en double dans l'agenda.*

    `publier` les évite pour un même créneau (même UID). S'il en reste, le titre lu a changé entre deux passages : normalisez-le dans votre parser.
12. *CalDAV : « agenda calevent introuvable ».*

    Utilisez le compte CalDAV de la squad (clé `caldav` de votre fichier d'identifiants), pas votre compte personnel.
13. *CalEvent ne propose aucun événement avec notre modèle.*

    Son parser ne connaît que les classes de référence : transmettez à l'intervenant la correspondance de vos classes (ex. `jour=date_cell`).

# 9. Glossaire

| | |
|---|---|
| **Annotation / label** | Information ajoutée à une donnée (ici : une boîte et une classe). |
| **Artefact** | Fichier produit par un run (poids du modèle, graphiques). |
| **Backbone / Neck / Head** | Les trois blocs d'un détecteur YOLO : extraction des motifs, combinaison des échelles, production des boîtes. |
| **Baseline** | Méthode simple servant de point de comparaison. |
| **Binarisation** | Transformer une image en noir et blanc strict ; « adaptative » : le seuil dépend de l'éclairage local. |
| **Bounding box** | Boîte englobante autour d'un objet. |

| | |
|---|---|
| **CalDAV** | Protocole standard de calendrier partagé (RFC 4791). |
| **Confiance** | Score (0 à 1) d'une détection ou d'un parsing. |
| **Epoch** | Un passage complet sur le jeu d'entraînement. |
| **Fine-tuning** | Reprendre un modèle pré-entraîné pour l'adapter à ses propres classes. |
| **Homographie** | Transformation qui redresse une vue en perspective (ici, une feuille photographiée de travers). |
| **IoU** | *Intersection over Union* : mesure de recouvrement entre deux boîtes. |
| **mAP** | *Mean Average Precision* : précision moyenne sur l'ensemble des classes. |
| **Morphologie mathématique** | Opérations sur une image binaire avec un petit motif (le *noyau*) ; l'ouverture efface les formes plus petites que le noyau. |
| **NMS** | *Non-Maximum Suppression* : ne garde qu'une boîte par objet, la plus confiante. |
| **OCR** | Reconnaissance optique de caractères. |
| **ONNX** | Format d'échange de modèles, indépendant du framework. |
| **Registry** | Catalogue versionné de modèles (MLflow). |
| **Run** | Un entraînement tracé dans MLflow. |
| **S3** | Protocole de stockage objet (RustFS en est une implémentation). |
| **Squad** | Équipe de 3 ou 4 étudiants. |
| **VEVENT** | Un événement au format iCalendar. |
| **YOLO** | *You Only Look Once* : détection d'objets en une seule passe. |

# 10. Ressources

- Modèle et suivi : Ultralytics (YOLO) docs.ultralytics.com · CVAT docs.cvat.ai · MLflow mlflow.org/docs/latest
- OCR et agenda : EasyOCR github.com/JaidedAI/EasyOCR · python-caldav caldav.readthedocs.io · Radicale radicale.org/v3.html
- iCalendar (RFC 5545) : datatracker.ietf.org/doc/html/rfc5545
- Support de cours : distribué par l'intervenant (dépôt flexistaff ou drive de l'École)
- Notebooks et modules fournis : dossier `ressources/` du dépôt flexistaff